TWINNET

A Local Simulation of Private AWS VPC Peering

Project domain: Cloud Networking / Virtual Private Cloud
Project format: Configuration design + executable Python logic simulation

Student details	
Name	Vaishnavi Subash Pillai
Register number	2114242440228
Class / Batch	Batch 1
Course	AWS
Submission date	10 November 2026

> **Scope statement:** This is a no-cloud simulation. No AWS account, EC2 instance, VPC, peering connection or paid cloud resource was created. The program checks the proposed routes and inbound security rules locally. It does not claim to be a live AWS deployment.

Contents

Abstract
1. Problem statement and objectives
2. Proposed architecture
3. Address and resource plan
4. Route-table design
5. Security-group design
6. Simulation method
7. Test cases and results
8. Failure analysis: route vs. security rule
9. Security and design observations
10. Limitations
11. Path to a live AWS deployment (planned, not performed)
12. Conclusion and learning outcome
13. Viva questions
14. Submission notes
References
Appendix A. Simulator source code
Appendix B. Glossary

Abstract

This project presents a private network design connecting two logically separate Virtual Private Clouds (VPCs) using VPC peering. VPC A hosts a public bastion and a private application host, while VPC B contains a second private application host. The application hosts communicate through private IPv4 addresses, and a bastion host is included for controlled administrative access.

Because this submission is designed to run without cloud resources, the network configuration is represented as data and evaluated by a Python simulator. The simulator checks route availability and destination inbound security rules. It demonstrates the expected baseline block, successful logical connectivity after peering routes are added, a controlled security-rule failure, and recovery after the rule is restored.

Key result: the simulation shows that peering alone does not create connectivity. Reachability required three things together: non-overlapping CIDR blocks, routes in both directions, and a matching inbound security rule on the destination.

1. Problem statement and objectives

An organization operates application workloads in two separate VPCs. The workloads need private communication without assigning public IP addresses to the application servers. The design must provide explicit routing, restricted inbound access and an administrative entry point.

Objectives

- Plan non-overlapping IPv4 CIDR ranges for two VPCs and their subnets.
- Design public and private subnet route tables.
- Specify a VPC peering connection and routes in both directions.

- Restrict inbound traffic using source-specific security-group rules.
- Simulate expected connectivity and failure scenarios locally.
- Explain the security and routing limitations of the design.

2. Proposed architecture

The design follows the supplied EWB workbook. VPC A uses 10.10.0.0/16 and VPC B uses 10.20.0.0/16. The ranges do not overlap, which is required for straightforward VPC peering.

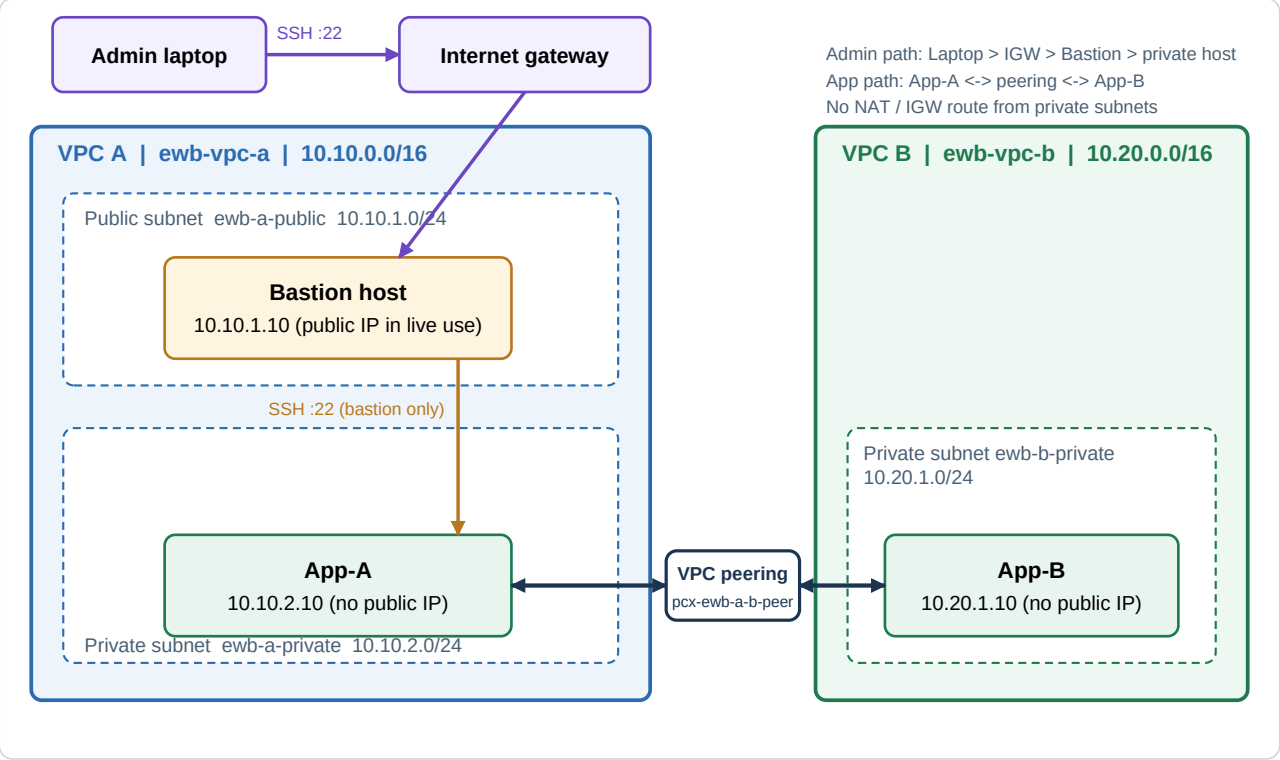

Figure 1. Logical architecture. Solid navy arrows show the peering path; purple and amber arrows show the administrative SSH path.

Administration path: Laptop to Bastion to private host. Application traffic path: App-A to VPC peering to App-B. The peering connection provides private routing between the VPCs; it does not make VPC B use VPC A as an internet gateway.

3. Address and resource plan

Resource / logical name	CIDR / private IPv4	Purpose
ewb-vpc-a	10.10.0.0/16	VPC A
ewb-a-public	10.10.1.0/24	Bastion subnet
ewb-a-private	10.10.2.0/24	App-A subnet
ewb-vpc-b	10.20.0.0/16	VPC B
ewb-b-private	10.20.1.0/24	App-B subnet
ewb-bastion	10.10.1.10	Administrative entry point; public IPv4 in live deployment
ewb-app-a	10.10.2.10	Private application host; no public IP
ewb-app-b	10.20.1.10	Private application host; no public IP

All names and addresses above are planned logical values from the workbook. Since this project does not provision AWS resources, AWS resource IDs are not applicable and are intentionally not invented.

4. Route-table design

Route table	Destination	Target	Associated subnet
ewb-rt-a-public	10.10.0.0/16	local	ewb-a-public
ewb-rt-a-public	10.20.0.0/16	VPC peering	ewb-a-public
ewb-rt-a-public	0.0.0.0/0	Internet gateway	ewb-a-public
ewb-rt-a-private	10.10.0.0/16	local	ewb-a-private
ewb-rt-a-private	10.20.0.0/16	VPC peering	ewb-a-private
ewb-rt-b-private	10.20.0.0/16	local	ewb-b-private
ewb-rt-b-private	10.10.0.0/16	VPC peering	ewb-b-private

Both VPCs need a route to the other VPC's CIDR. The return route is equally important: without it, replies cannot reliably travel back. The local route is retained in each route table. The public route table also has a default route to the internet gateway for the bastion subnet.

Design note: the private route tables have no 0.0.0.0/0 route, so App-A and App-B cannot reach the internet. This is intentional for this project; a NAT gateway would be needed if the hosts had to download updates.

5. Security-group design

Security group	Inbound source	Protocol / port	Reason
ewb-sg-bastion	Administrator public IP /32	TCP 22 (SSH)	Restrict admin access to one source IP
ewb-sg-app-a	10.10.1.10/32	TCP 22 (SSH)	SSH from bastion only
ewb-sg-app-a	10.20.1.10/32	ICMP Echo	Allow ping from App-B
ewb-sg-app-b	10.10.1.10/32	TCP 22 (SSH)	SSH from bastion only
ewb-sg-app-b	10.10.2.10/32	ICMP Echo	Allow ping from App-A

The /32 suffix identifies one IPv4 host. Narrow source rules reduce unnecessary exposure. The administrator's actual public IP is intentionally left as a placeholder rather than guessed.

6. Simulation method

The Python program `TwinNet_VPC_Peering_Simulator.py` represents route tables and security-group inbound rules as dictionaries. For each simulated packet it:

- Finds the route table associated with the source host.
- Selects the most-specific (longest-prefix) route matching the destination IP.
- Checks whether the selected route points to the simulated peering connection.
- Checks whether the destination security group allows the source IP and protocol.
- Reports PASS or BLOCKED based on both checks.

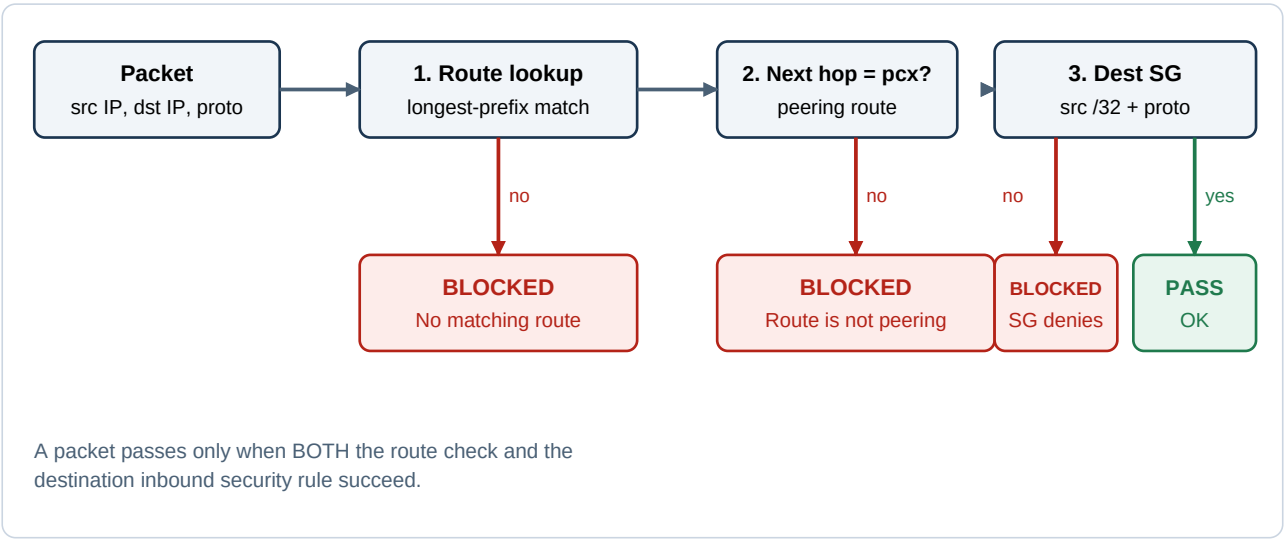

Figure 2. Packet-evaluation logic used by the simulator.

The simulation is deterministic and uses Python's standard library only. It does not open sockets, send ICMP packets, connect to AWS or require third-party packages. The full source is in Appendix A.

7. Test cases and results

#	Test	Expected result	Observed	Reason
1	App-A to App-B before peering routes	BLOCKED	**BLOCKED**	No route to remote VPC CIDR
2	App-A to App-B after routes	PASS	**PASS**	Forward route and App-B ICMP rule exist
3	App-B to App-A after routes	PASS	**PASS**	Return route and App-A ICMP rule exist
4	App-A to App-B after removing App-B ICMP rule	BLOCKED	**BLOCKED**	Destination inbound rule denies ICMP
5	App-A to App-B after restoring rule	PASS	**PASS**	Route and inbound permission restored

Verified local simulator output: The output below was generated by running the included Python file in a local Python environment on 10 November 2026. It is simulation output, not an AWS console or EC2 terminal capture.

```
============================================================
TWINNET | AWS VPC PEERING - LOCAL LOGIC SIMULATION
No AWS resources are created; no real packets are transmitted.
============================================================
1) BASELINE BEFORE PEERING
App-A to App-B before peering: BLOCKED
  Packet: 10.10.2.10 -> 10.20.1.10 (ICMP)
  Route: No matching route
  Destination inbound rule: ALLOW
2) CONNECTIVITY AFTER PEERING ROUTES
Forward: App-A to App-B: PASS
  Packet: 10.10.2.10 -> 10.20.1.10 (ICMP)
  Route: A-private: 10.20.0.0/16 -> pcx-ewb-a-b-peer
  Destination inbound rule: ALLOW
```

```
Reverse: App-B to App-A: PASS
  Packet: 10.20.1.10 -> 10.10.2.10 (ICMP)
  Route: B-private: 10.10.0.0/16 -> pcx-ewb-a-b-peer
  Destination inbound rule: ALLOW
3) CONTROLLED SECURITY-GROUP FAILURE
App-A to App-B after removing ICMP rule: BLOCKED
  Packet: 10.10.2.10 -> 10.20.1.10 (ICMP)
  Route: A-private: 10.20.0.0/16 -> pcx-ewb-a-b-peer
  Destination inbound rule: DENY
4) RECOVERY AFTER RESTORING ICMP RULE
App-A to App-B after restoring ICMP rule: PASS
  Packet: 10.10.2.10 -> 10.20.1.10 (ICMP)
  Route: A-private: 10.20.0.0/16 -> pcx-ewb-a-b-peer
  Destination inbound rule: ALLOW
```

```
5) PRIVATE-HOST ADDRESS CHECK
  App-A private address: 10.10.2.10 | Public IP: NONE (design)
  App-B private address: 10.20.1.10 | Public IP: NONE (design)
SIMULATION SUMMARY
  Baseline blocked: YES (expected)
  Forward and reverse logical checks passed: True
  Removing inbound ICMP blocked traffic: True
  Restoring inbound ICMP restored traffic: True
Note: These are deterministic configuration checks, not AWS or OS-level ping results.
```

8. Failure analysis: route vs. security rule

Tests 1 and 4 both end in BLOCKED, but for different reasons. The simulator prints both checks so the cause is visible:

Scenario	Route check	Inbound rule check	Diagnosis
Test 1 (before peering)	No matching route	ALLOW	Routing problem: add the peering routes
Test 4 (rule removed)	Route to pcx found	DENY	Security-group problem: restore the ICMP rule

This mirrors real troubleshooting: when two hosts cannot talk, check the route table first, then the destination security group, and only then look at network ACLs or the operating-system firewall.

9. Security and design observations

- The two VPC CIDR blocks are non-overlapping.
- Application instances are designed without public IPv4 addresses.
- The bastion is the only planned public administrative entry point.
- SSH access to private hosts is limited to the bastion's private address.
- ICMP rules are host-specific and used for connectivity testing.
- VPC peering is non-transitive; a peering link between A and B does not automatically connect a third VPC.
- VPC peering does not provide transitive internet access or automatically configure routes and security groups.
- Private subnets have no default route, so the application hosts have no outbound internet access.

10. Limitations

This project validates the logic of the proposed route and security configuration. It does not prove AWS resource creation, real packet delivery, SSH login, AWS route propagation, network ACL behavior,

operating-system firewall behavior or AWS billing/cleanup. A live deployment would still need the actual AWS console steps and evidence required by the original workbook. The simulator's PASS result means the model's route and inbound-rule checks passed; it must not be described as a successful real ping.

Additional simplifications: the model checks only ICMP and TCP/22 rule matching, ignores outbound (egress) rules, which allow all traffic by default in AWS, and does not model stateful return traffic.

11. Path to a live AWS deployment (planned, not performed)

If the project is later extended, the simulated design maps to the following console steps. None of these were carried out for this submission.

Phase	Planned action	Evidence to capture
Build	Create ewb-vpc-a and ewb-vpc-b; add the three subnets; attach an internet gateway to VPC A	VPC and subnet list with CIDRs
Peer	Create the peering connection from VPC A to VPC B and accept it	Peering status: Active
Route	Add the 10.20.0.0/16 and 10.10.0.0/16 peering routes to the three route tables	Route-table screenshots
Secure	Create ewb-sg-bastion, ewb-sg-app-a and ewb-sg-app-b with the Section 5 rules	Security-group rule screenshots
Launch	Launch bastion, App-A and App-B with the planned private IPs; no public IP on app hosts	Instance list
Validate	SSH laptop to bastion to App-A; ping App-A to App-B and back	Terminal captures
Clean up	Terminate instances, delete the peering connection, route entries, security groups, subnets, internet gateway and VPCs; confirm no billable resources remain	Empty resource lists

12. Conclusion and learning outcome

The project demonstrates how private connectivity between two VPCs depends on three coordinated elements: non-overlapping address spaces, routes in both directions, and security rules that permit the intended traffic. It also shows that a missing route and a missing inbound permission produce different logical failure causes. The local simulator makes these dependencies testable without creating cloud resources.

The main learning outcome is that network reachability is not established by peering alone. The peering connection, route tables and security controls must all be configured consistently.

13. Viva questions

Q. What is VPC peering?

A. A private network connection between two VPCs that allows routing between their private IP ranges.

Q. Why must CIDR ranges not overlap?

A. Overlapping address ranges make destination routing ambiguous and prevent normal peering configuration.

Q. Why are routes needed on both sides?

A. Traffic needs a forward path and a return path.

Q. What is a bastion host?

A. A controlled administrative host used as an entry point to reach private instances.

Q. Why use /32 in security rules?

A. It represents one IPv4 address, limiting the source to a single host.

Q. Does peering automatically allow traffic?

A. No. Routes and security rules must also allow the traffic.

Q. Is this a live AWS deployment?

A. No. It is a local logic simulation based on the AWS design; no AWS resources were created.

Q. What does 'most-specific route' mean?

A. When several routes match a destination, the one with the longest prefix (for example /24 over /16) is used.

Q. Why is VPC peering called non-transitive?

A. A VPC cannot reach a third VPC through the peered VPC; each pair needs its own peering connection.

Q. Why do the private hosts have no internet access here?

A. Their route tables have no default route; a NAT gateway would be required to add outbound access.

14. Submission notes

The supplied workbook asks for one PDF named YourName_EWB_VPC_Peering.pdf, with student details, architecture, inventory, route tables, security rules, test evidence, explanation and cleanup evidence. This no-cloud version cannot provide actual AWS IDs, live peering status, SSH/ping screenshots or AWS cleanup evidence. Before submitting, ask the faculty whether a clearly labelled simulation-based alternative is accepted.

If accepted, submit this report PDF and, if the submission portal permits supporting files, include the Python simulator file. If only one file is accepted, submit the PDF; the source code is already included as Appendix A.

Do not relabel the simulated tests as live AWS tests or add fabricated resource IDs/screenshots.

References

Primary project specification: EWB Courses, AWS VPC Peering Student Project Workbook, October 2026, especially the architecture, resource plan, validation, deliverables and cleanup sections. The configuration names and IP plan in this report follow that workbook.

Additional technical background: AWS documentation on VPC peering, route tables and security groups should be consulted if the project is later extended into a live deployment.

Appendix A. Simulator source code

File: `TwinNet_VPC_Peering_Simulator.py` (Python 3, standard library only). Run with `python TwinNet_VPC_Peering_Simulator.py`.

```
#!/usr/bin/env python3
"""TwinNet | Local logic simulation of AWS VPC peering.

No AWS resources are created and no real packets are sent. Route tables and
security-group inbound rules are plain dictionaries; each simulated packet is
checked for (1) a matching route and (2) a destination inbound rule.
Standard library only.
"""
import ipaddress
from copy import deepcopy

PEER = "pcx-ewb-a-b-peer"

HOSTS = {
    "bastion": {"ip": "10.10.1.10", "rt": "A-public", "sg": "ewb-sg-bastion"},
    "app-a": {"ip": "10.10.2.10", "rt": "A-private", "sg": "ewb-sg-app-a"},
    "app-b": {"ip": "10.20.1.10", "rt": "B-private", "sg": "ewb-sg-app-b"},
}

# Baseline: local routes only (no peering routes yet)
ROUTES_BASELINE = {
    "A-public": [("10.10.0.0/16", "local"), ("0.0.0.0/0", "igw")],
    "A-private": [("10.10.0.0/16", "local")],
    "B-private": [("10.20.0.0/16", "local")],
}

PEERING_ROUTES = {
    "A-public": ("10.20.0.0/16", PEER),
    "A-private": ("10.20.0.0/16", PEER),
    "B-private": ("10.10.0.0/16", PEER),
}

SG_RULES = {
    "ewb-sg-app-a": [("10.10.1.10/32", "tcp", 22), ("10.20.1.10/32", "icmp", None)],
    "ewb-sg-app-b": [("10.10.1.10/32", "tcp", 22), ("10.10.2.10/32", "icmp", None)],
}


def best_route(routes, dst):
    """Longest-prefix match, as a VPC route table does."""
    dst = ipaddress.ip_address(dst)
    hits = [(ipaddress.ip_network(c), t) for c, t in routes if dst in ipaddress.ip_network(c)]
    return max(hits, key=lambda h: h[0].prefixlen) if hits else None


def sg_allows(sg, src, proto, port):
    src = ipaddress.ip_address(src)
    for cidr, p, prt in SG_RULES.get(sg, []):
        if src in ipaddress.ip_network(cidr) and p == proto and (prt is None or prt == port):
            return True
    return False


def check(routes, src, dst, proto="icmp", port=None):
    s, d = HOSTS[src], HOSTS[dst]
    hit = best_route(routes[s["rt"]], d["ip"])
    route_ok = bool(hit) and hit[1] == PEER
    sg_ok = sg_allows(d["sg"], s["ip"], proto, port)
    return {
        "src": src, "dst": dst, "packet": f"{s['ip']} -> {d['ip']} ({proto.upper()})",
        "route": f"{s['rt']}: {hit[0]} -> {hit[1]}" if hit else "No matching route",
        "route_ok": route_ok, "sg": "ALLOW" if sg_ok else "DENY",
```

```
        "result": "PASS" if route_ok and sg_ok else "BLOCKED",
    }


def show(title, r):
    print(f"{title}: {r['result']}")
    print(f"  Packet: {r['packet']}")
    print(f"  Route: {r['route']}")
    print(f"  Destination inbound rule: {r['sg']}")


def main():
    bar = "=" * 62
    print(bar)
    print("TWINNET | AWS VPC PEERING - LOCAL LOGIC SIMULATION")
    print("No AWS resources are created; no real packets are transmitted.")
    print(bar)

    print("1) BASELINE BEFORE PEERING")
    base = check(ROUTES_BASELINE, "app-a", "app-b")
    show("App-A to App-B before peering", base)

    routes = deepcopy(ROUTES_BASELINE)
    for table, entry in PEERING_ROUTES.items():
        routes[table].append(entry)

    print("2) CONNECTIVITY AFTER PEERING ROUTES")
    fwd = check(routes, "app-a", "app-b")
    rev = check(routes, "app-b", "app-a")
    show("Forward: App-A to App-B", fwd)
    show("Reverse: App-B to App-A", rev)

    print("3) CONTROLLED SECURITY-GROUP FAILURE")
    removed = ("10.10.2.10/32", "icmp", None)
    SG_RULES["ewb-sg-app-b"].remove(("10.10.2.10/32", "icmp", None))
    fail = check(routes, "app-a", "app-b")
    show("App-A to App-B after removing ICMP rule", fail)

    print("4) RECOVERY AFTER RESTORING ICMP RULE")
    SG_RULES["ewb-sg-app-b"].append(removed)
    rec = check(routes, "app-a", "app-b")
    show("App-A to App-B after restoring ICMP rule", rec)

    print("5) PRIVATE-HOST ADDRESS CHECK")
    for h in ("app-a", "app-b"):
        print(f"  {h.title().replace('-', '-')} private address: {HOSTS[h]['ip']} | Public IP: NONE (design)")

    print("SIMULATION SUMMARY")
    print(f"  Baseline blocked: {'YES (expected)' if base['result'] == 'BLOCKED' else 'NO'}")
    print(f"  Forward and reverse logical checks passed: {fwd['result'] == rev['result'] == 'PASS'}")
    print(f"  Removing inbound ICMP blocked traffic: {fail['result'] == 'BLOCKED'}")
    print(f"  Restoring inbound ICMP restored traffic: {rec['result'] == 'PASS'}")
    print("Note: These are deterministic configuration checks, not AWS or OS-level ping results.")


if __name__ == "__main__":
    main()
```

Appendix B. Glossary

Term	Meaning
VPC	Virtual Private Cloud: an isolated virtual network in AWS.
CIDR	Notation for an IP range, e.g. 10.10.0.0/16.
Subnet	A slice of a VPC's address range placed in one availability zone.
Route table	Rules that decide where traffic for a destination range is sent.
VPC peering	A private, non-transitive link between two VPCs.
Security group	A stateful, instance-level firewall with allow rules.
Bastion host	A hardened public host used to administer private hosts.
ICMP echo	The message type used by ping.

Term	Meaning
Internet gateway	The VPC component that gives public subnets internet access.